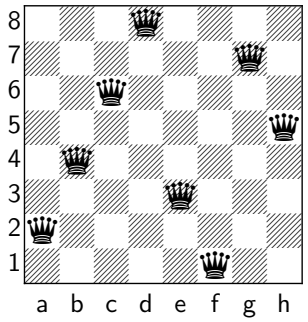
Grau de Matemàtiques (UAB) *Curs 2023-2024*
Eines informàtiques per a les matemàtiques

# Treball: El problema de les 8 reines.

## Una mica d'història

L'anomenat *Problema de les 8 reines* fou proposat el setembre de 1848 pel jugador bavarès d'escacs Max Friedrich William Bezzel (1824-1871) a la revista berlinesa *Schachzeitung* (Periòdic d'Escacs) sota el pseudònim de *Schachfreund* (Amic dels escacs). El problema consisteix a trobar totes les possibles combinacions de 8 reines situades en un tauler d'escacs de tal manera que no s'amenacin entre si, així doncs, 2 reines no poden estar alhora en la mateixa fila, columna o diagonal. En el següent esquema apareix una possible solució:



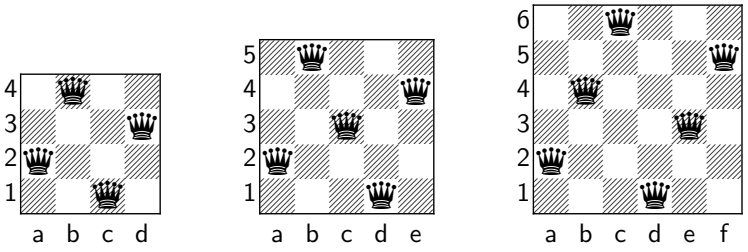

Aquesta solució es pot codificar amb el vector $(2, 4, 6, 8, 3, 1, 7, 5) \in \mathbb{N}^8$.
El mateix Bezzel no va poder resoldre completament el problema. L'any 1850, Franz Nauck va donar el número 92 correcte al *Leipziger Illustrirten Zeitung* (Diari Il·lustrat de Leipzig). Més tard es va demostrar que no hi havia més solucions. Posteriorment el problema es va estendre a trobar el nombre de solucions possibles en un tauler de dimensió arbitrària $n \times n$ i fins i tot en Carl Friedrich Gauss s'en va ocupar d'aquest problema. Avui dia, el problema de les $n-$reines és un problema obert, encara no es coneix el nombre total de solucions per un tauler d'escacs de dimensió arbitrària $n \times n$, només es coneix la solució $\forall n \leq 27$. Per exemple, no existeix cap solució per $n = 2$ o per $n = 3$, existeix una única solució per $n = 1$, dues solucions per $n = 4$ i quatre solucions per $n = 6$.
Direm que dues solucions són essencialment diferents si no existeix una simetria que transformi una en l'altra. Aquesta observació ens permet reduïr el temps d'execució i la capacitat de memòria utilitzada per l'ordinador. De fet, en el tauler d'escacs actua el grup de simetries del quadrat o grup diedral de 8 elements $D_{2\cdot 4}$. En la següent taula hi ha el valors coneguts actualment (2024) del nombre total de solucions i de solucions essencialment diferents.

| n | Solucions essencialment diferents | Solucions totals | Any |
|---|---|---|---|
| 1 | 1 | 1 | |
| 2 | 0 | 0 | |
| 3 | 0 | 0 | |
| 4 | 1 | 2 | |
| 5 | 2 | 10 | |
| 6 | 1 | 4 | |
| 7 | 6 | 40 | |
| 8 | 12 | 92 | |
| 9 | 46 | 352 | |
| 10 | 92 | 724 | |
| 11 | 341 | 2.680 | |
| 12 | 1.787 | 14.200 | 1969 |
| 13 | 9.233 | 73.712 | |
| 14 | 45.752 | 365.596 | |
| 15 | 285.053 | 2.279.184 | |
| 16 | 1.846.955 | 14.772.512 | |
| 17 | 11.977.939 | 95.815.104 | |
| 18 | 83.263.591 | 666.090.624 | |
| 19 | 621.012.754 | 4.968.057.848 | |
| 20 | 4.878.666.808 | 39.029.188.884 | |
| 21 | 39.333.324.973 | 314.666.222.712 | |
| 22 | 336.376.244.042 | 2.691.008.701.644 | |
| 23 | 3.029.242.658.210 | 24.233.937.684.440 | |
| 24 | 28.439.272.956.934 | 227.514.171.973.736 | |
| 25 | 275.986.683.743.434 | 2.207.893.435.808.352 | 2005 |
| 26 | 2.789.712.466.510.289 | 22.317.699.616.364.044 | 2009 |
| 27 | 29.363.495.934.315.694 | 234.907.967.154.122.528 | 2016 |

El darrer valor per $n = 27$ es va calcular el 2016 i a l'any següent la Universitat de Saint Andrews (Escòcia) va oferir un millió de dòlars a qui resolgui el problema de les $n-$reines.
En la taula anterior s'observa que el nombre total de solucions augmenta exponencialment al crèixer el valor de $n$, i per tant cal esperar que també creixi exponencialment el temps necessari per resoldre el següent cas, i possiblement la memòria utilitzada per l'ordinador.

# 1 Algunes solucions

Tot i que el nombre total de solucions només es coneix per tot $n \leq 27$, és possible trobar alguna solució per taulers d'escacs de dimensió superior a 28. Així, per exemple, l'any 1991 Socic i Gu van trobar un algorisme que calculava, en un temps raonable, algunes solucions per taulers d'escacs de dimensió $n < 3.000.000$. Algunes solucions per valors baixos de $n$ són les següents:




(2,4,1,3) (2,5,3,1,4) (2,4,6,1,3,5)

El primer vector $\vec{v} = (2, 4, 1, 3)$ codifica una solució del tauler $4 \times 4$. Si comencem a comptar les files i columnes a partir del vèrtex inferior esquerra del tauler tenim una reina a la fila 2 de la 1a columna, una reina a la fila 4 de la 2a columna, una reina a la fila 1 de la 3a columna i una reina a la fila 3 de la 4a columna.

Així doncs, com en una mateixa columna no pot haber-hi dues o més reines, les solucions de $n-$reines en un tauler d'escacs de dimensions $n \times n$ es poden codificar amb un vector $\vec{v} = (v_1, \ldots, v_n) \in \mathbb{N}^n$. De fet $\vec{v} \in [1, \ldots, n]^n$, on el valor de cada component $v_i$ representa en quina fila està situada la reina de la columna $i-$èsima. El cardinal del conjunt $[1, \ldots, n]^n$ és $n^n$ que coincideix amb les variacions amb repetició del conjunt de $n$ elements $\{1, \ldots, n\}$ agafats de $n$ en $n$. Aquesta és una fita molt grollera del nombre total de solucions al problema de les $n-$reines.

$$\text{nombre solucions n-reines} \leq VR_n^n = n^n$$

Els valors d'aquesta successió de nombres naturals creix molt ràpidament

$$\{n^n\}_{n\in\mathbb{N}} = \{1, 4, 27, 256, 3.125, 46.656, 823.543, 16.777.216, \ldots\}$$

En canvi el nombre total de solucions al problema de les $n-$reines és una successió amb valors molt més baixos

$$\{\text{nombre solucions n-reines}\}_{n\in\mathbb{N}} = \{1, 0, 0, 2, 10, 4, 40, 92, \ldots\}$$

Per altra banda, com en cada fila només pot haver-hi una reina, aleshores tots els valors de les components del vector $\vec{v}$ són diferents, $v_i \neq v_j$, $1 \leq i, j \leq n$. Per tant, els valors del vector $\vec{v}$ són una permutació del conjunt $\{1, \ldots, n\}$, és a dir, es poden pensar com elements del grup simètric de $n$ elements, $\vec{v} \in S_{\{1,\ldots,n\}} = S_n$. El cardinal d'aquest conjunt és $n!$ que coincideix amb les permutacions (sense repetició) del conjunt de $n$ elements $\{1, \ldots, n\}$. Aquesta és encara una fita força grollera del nombre total de solucions al problema de les $n-$reines.

$$\text{nombre solucions n-reines} \leq P_n = n! \leq VR_n^n = n^n$$

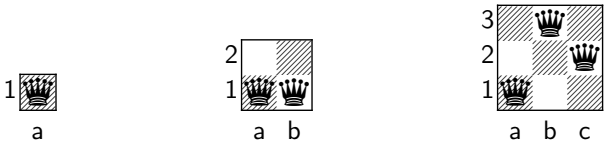
Els valors de la successió $P_n$ també tenen un creixement ràpid però més lent que els valors de $VR_n^n$

$$\{n!\}_{n\in\mathbb{N}} = \{1, 2, 6, 24, 120, 720, 5.040, 40.320, 362.880, 362.8800, 39.916.800, \ldots\}$$

Finalment, com dues reines no poden estar en una mateixa diagonal, això es tradueix en la darrera condició $\mid v_i - v_j \mid \neq \mid i - j \mid$, $1 \leq i, j \leq n$ que cal imposar a un vector $\vec{v} \in [1, \ldots, n]^n$ candidat a solució al problema de les $n$-reines.

1. Fes una funció anomenada `es_solucio` que decideixi si un vector $\vec{v} \in [1, \ldots, n]^n$ que codifica un conjunt de $n-$reines distribuïdes en $n$-columnes diferents d'un tauler d'escacs de dimensions $n \times n$ és solució del problema de les $n$-reines.

   Per exemple el vector (1) és solució i en canvi els vectors $(1, 1)$ i $(1, 3, 2)$ no són solució.




(1) (1,1) (1,3,2)

   Quin és el tauler màxim amb el que el teu codi decideix si un vector és o no solució? i quin és el temps que triga?

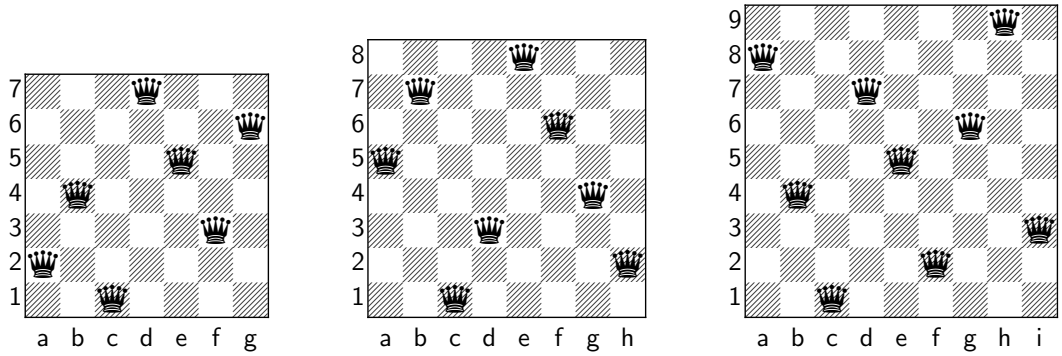
# 2 Cerca aleatòria d'una solució

Una manera de trobar solucions a un problema és treballar amb l'atzar.

2. Fes una segona funció anomenada `vector_aleatori` que donada la dimensió $n$ d'un tauler d'escacs $n \times n$ generi un vector aleatori $\vec{v} = (v_1, \ldots, v_n) \in [1, \ldots, n]^n$. Indicació: per generar valors aleatoris pots utilitzar la funció *rand*() que es troba dins la llibreria *stdlib.h*.

   Comprova després si aquest vector que has generat és una solució aleatòria al problema de les $n-$reines. Fes un bucle amb aquesta funció fins que trobis una solució al problema de les $n-$reines. Quina és la dimensió més gran d'un tauler d'escacs on pots trobar una solució utilitzant aquest mètode aleatori? i quin és el temps que triga en trobar una solució aleatòria?

   Per exemple, tres solucions calculades a l'atzar del problema de les $n-$reines són els vectors $\vec{u} = (2, 4, 1, 7, 5, 3, 6) \in \mathbb{Z}^7$, $\vec{v} = (5, 7, 1, 3, 8, 6, 4, 2) \in \mathbb{Z}^8$ i $\vec{w} = (8, 4, 1, 7, 5, 2, 6, 9, 3) \in \mathbb{Z}^9$ per $n = 7$, $n = 8$ i $n = 9$ respectivament.



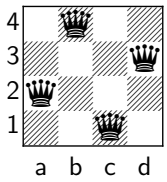

(2,4,1,7,5,3,6) (5,7,1,3,8,6,4,2) (8,4,1,7,5,2,6,9,3)

# 3 Cerca aleatòria de totes les solucions

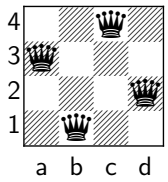
3. Fes una tercera funció anomenada `totes_les_solucions_aleatories` que donada la dimensió $n$ d'un tauler d'escacs $n \times n$ generi vectors aleatoris i pari quan trobi totes les solucions al problema de les $n-$reines. El conjunt de totes les solucions s'ha de desar en un fitxer de sortida anomenat `sortida_aleatori3_nxn.txt`. En cada fila d'aquest fitxer hi haurà una solució diferent al problema de les $n-$reines codificada per un vector de $n$ components, les components del vector es separaran per espais en blanc.

   Quin és el valor de la dimensió $n$ més gran d'un tauler d'escacs on pots trobar totes les solucions utilitzant aquest mètode aleatori? i quin és el temps que triga en trobar aleatòriament totes les solucions?

   Per exemple totes les solucions del cas $n = 4$ són:




(2,4,1,3)




(3,1,4,2)

   En aquest cas, el fitxer `sortida_aletori3_4x4.txt` ha de tenir la següent informació:

```
2 4 1 3
3 1 4 2
```

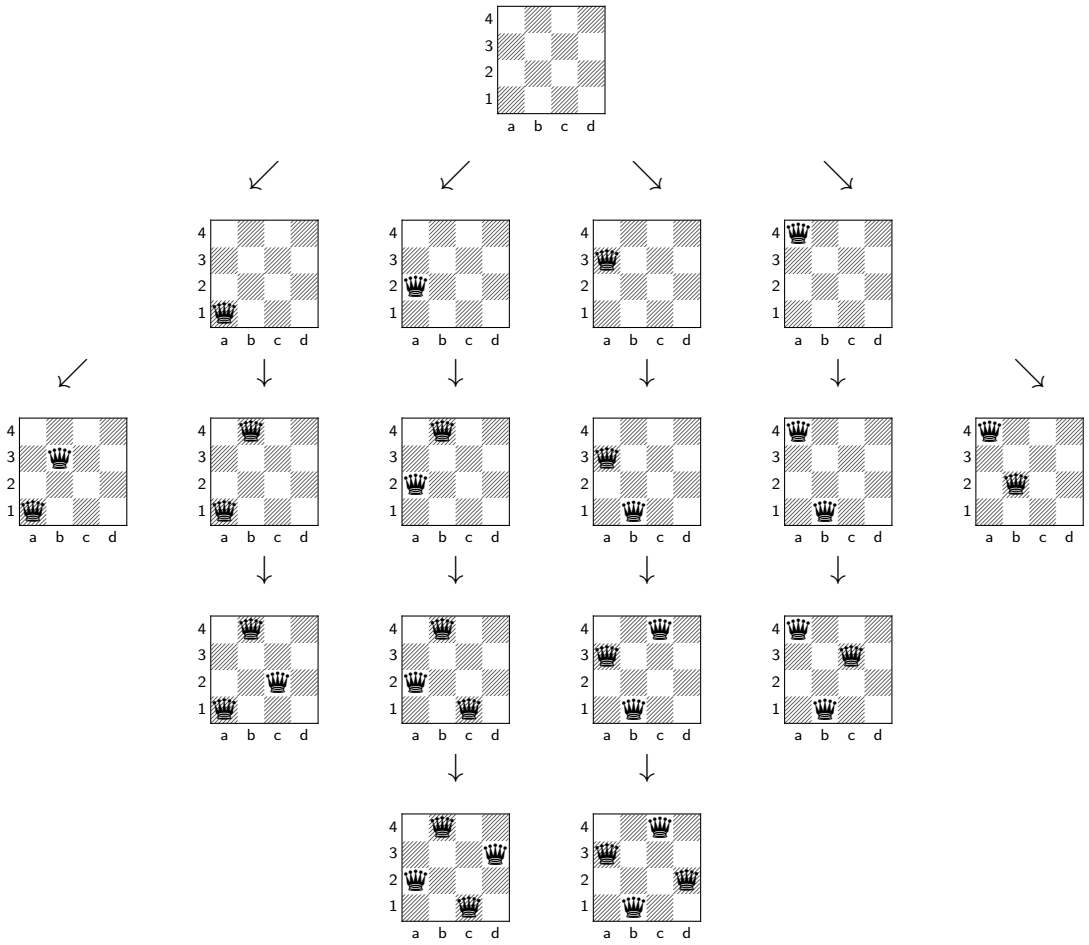
   o bé:

```
3 1 4 2
2 4 1 3
```

   Pel càlcul d'aquest exemple ($n = 4$) el programa hauria de trigar menys de 1 segon.

# 4 Totes les solucions. Mètode Backtracking

Una manera de resoldre el problema de les $n-$reines és recorrer les columnes del tauler d'escacs d'esquerra a dreta i per cada columna recorrer les files d'abaix a dalt, marcant les possibles solucions. Veiem-ho pel cas $n = 4$.



Aquesta estratègia per trobar totes les solucions s'anomena Backtracking o cerca en retrocès. El terme *backtrack* per referir-se a aquesta estratègia, va ser creat pel matemàtic nord-americà D.H.Lehmer a la dècada dels anys 50 del segle passat. Aquest exemple concret que hem vist pel cas $n = 4$ es pot aplicar a qualsevol tauler de dimensió $n \times n$ i si tinguèsim temps infinit podríem saber quin el nombre total de solucions al problema de les $n-$reines per qualsevol valor arbitrari de $n \in \mathbb{N}$.

4. Crea una funció anomenada `backtracking` que resolgui el problema de les $n-$reines utilitzant l'algorisme de Backtracking segons el següent pseudocodi recursiu:

```
funcio backtracking(tauler,columna,k)
       recorre files de 1 a n
         si (acceptable) aleshores
           si (solucio) aleshores
              tractar_solucio
              k++
            sino
             backtracking(tauler,columna+1,k)
            fi si
           fi si
         fi si
```

```
       fi recorre
     return k
fi funcio
```

on `tauler` és un vector de $n$ components on s'aniran desant les diferents solucions al problema de les $n-$reines i el paràmetre `k` ens indica quina és la $k-$èsima solució.

En el tractament de la solució, la funció `bactracking` anirà desant en un fitxer de sortida anomenat `sortida_bactrack_nxn.txt` totes les solucions. En cada línia d'aquest fitxer hi haurà una solució diferent, les components dels vectors solució estaran separades només per un espai en blanc. Aquesta funció també imprimirà per pantalla la mida del tauler, l'algorisme utilitzat i el nombre total de solucions al problema de les $n-$reines.

Per exemple si $n = 5$ el fitxer `sortida_bactrack_5x5.txt` ha de contenir la següent informació:

```
5 2 4 1 3
5 3 1 4 2
3 5 2 4 1
2 5 3 1 4
4 2 5 3 1
1 3 5 2 4
4 1 3 5 2
1 4 2 5 3
2 4 1 3 5
3 1 4 2 5
```

o bé una permutació de les files dels vectors de 5 components, i la impressió per pantalla ha de ser:

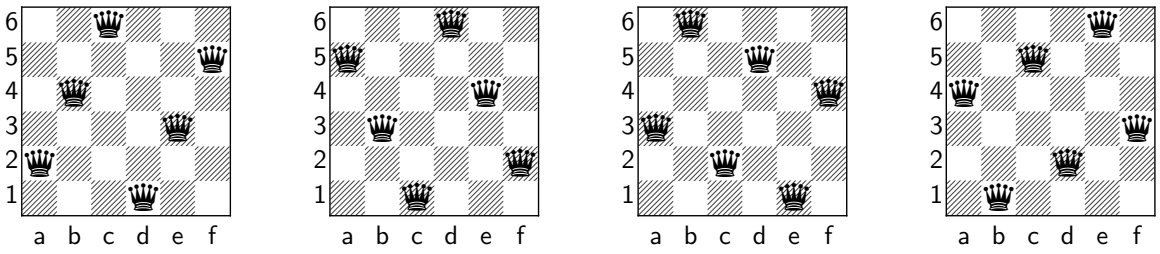
```
n=5 (backtracking)
10 solucions
```

Quin és el valor de la dimensió $n$ més gran d'un tauler d'escacs on pots aplicar aquest algorisme de Backtracking? i quin és el temps que triga en fer-ho?

És recomanable treballar amb memòria dinàmica per tractar les solucions dels taulers de mida més gran.

# 5 Quantes solucions hi ha per cada casella?

En els apartats anterior tens descrites les solucions al problema de les 4 reines, observa que en totes les solucions les reines apareixen en la perifèria del tauler i en cap solució apareixen reines en les diagonals. Per $n = 6$ només existeixen les 4 solucions següents:



Observa que per cada casella que no estigui en les diagonals hi ha una única solució. Això són característiques particulars d'aquests valors de `n`. En general, per a cada casella fixada, hi ha una certa quantitat de solucions que tenen allà una reina.

5. Fes una cinquena funció anomenat `matriu_compta_solucions` que desi en una matriu el nombre de solucions que hi ha per cada casella. Per valors baixos de $n$ els valors d'aquesta matriu són:

$$\begin{pmatrix} 1 \end{pmatrix} \quad \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \end{pmatrix}$$

$n = 1$ $n = 2$ $n = 3$ $n = 4$ $n = 5$

$$\begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 4 & 7 & 6 & 6 & 6 & 7 & 4 \\ 7 & 4 & 6 & 6 & 6 & 4 & 7 \\ 6 & 6 & 6 & 4 & 6 & 6 & 6 \\ 6 & 6 & 4 & 8 & 4 & 6 & 6 \\ 6 & 6 & 6 & 4 & 6 & 6 & 6 \\ 7 & 4 & 6 & 6 & 6 & 4 & 7 \\ 4 & 7 & 6 & 6 & 6 & 7 & 4 \end{pmatrix} \quad \begin{pmatrix} 4 & 8 & 16 & 18 & 18 & 16 & 8 & 4 \\ 8 & 16 & 14 & 8 & 8 & 14 & 16 & 8 \\ 16 & 14 & 4 & 12 & 12 & 4 & 14 & 16 \\ 18 & 8 & 12 & 8 & 8 & 12 & 8 & 18 \\ 18 & 8 & 12 & 8 & 8 & 12 & 8 & 18 \\ 16 & 14 & 4 & 12 & 12 & 4 & 14 & 16 \\ 8 & 16 & 14 & 8 & 8 & 14 & 16 & 8 \\ 4 & 8 & 16 & 18 & 18 & 16 & 8 & 4 \end{pmatrix}$$

$n = 6$ $n = 7$ $n = 8$

Quin és el valor de la dimensió $n$ més gran d'un tauler d'escacs on pots trobar aquestes matrius que t'indiquen el nombre de solucions per cada casella? i quin és el temps que triga en trobar-la?

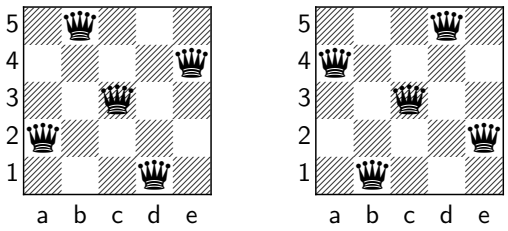
Observa que si tens una solució al problema de les $n-$reines, aleshores al fer simetries axials respecte les diagonals del taulell ($d_1$, $d_2$), simetries axials respecte els eixos centrals horitzontal ($s_h$) o vertical ($s_v$), o fer rotacions de 90 ($r$), 180 ($r^2$), o 270 ($r^3$) graus (simetries centrals), la configuració de les reines resultant també és una solució del mateix taulell d'escacs $n \times n$. El conjunt d'aquestes 7 isometries, juntament amb la isometria identitat tenen de manera natural una estructura de grup, on la operació és la composició d'isometries. Aquest grup s'anonema grup de simetries del quadrat o grup diedral de 8 elements. La notació d'aquest grup és $D_{2\cdot 4}$. Per referir-nos al fet anterior, direm que existeix una acció del grup diedral en el conjunt de totes les solucions o que el grup diedral $D_{2\cdot 4}$ actua en el conjunt de totes les solucions d'un taulell d'escacs $n \times n$. L'acció d'un element del grup diedral a una solució pot donar una solució diferent o no.

En el cas $n = 4$ les solucions són invariants per rotacions de 0, 90, 180 i 270 graus, és a dir, són invariants per l'acció d'aquests 4 elements del grup diedral de 8 elements $D_{2\cdot 4}$. La segona solució es pot obtenir fent una simetria respecte qualsevol de les diagonals o respecte l'eix central vertical o horitzontal. Així doncs la primera solució genera 2 solucions al aplicar una rotació o una simetria axial. En aquest cas direm que aquesta solució té una òrbita de 2 elements per l'acció del grup diedral $D_{2\cdot 4}$.

En el cas $n = 6$ les 4 solucions es poden obtenir per rotacions o simetries de la primera solució, en aquest cas la primera solució (i de fet totes les solucions) tenen una òrbita de 4 elements.

Per $n = 5$ existeixen 10 solucions, dos d'aquestes solucions tenen una òrbita de 2 elements i les 8 restants tenen una òrbita de 8 elements.

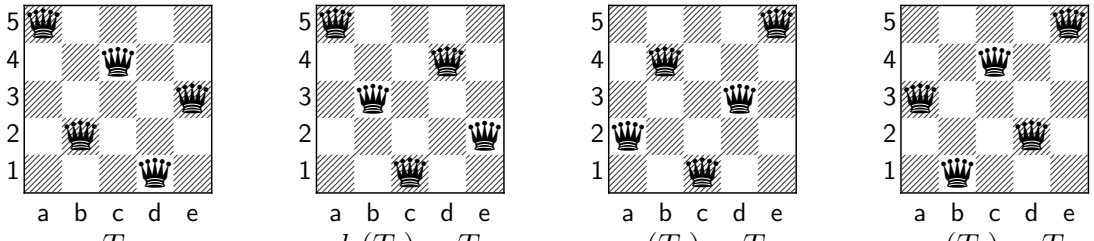
Amb òrbita de 2 elements:



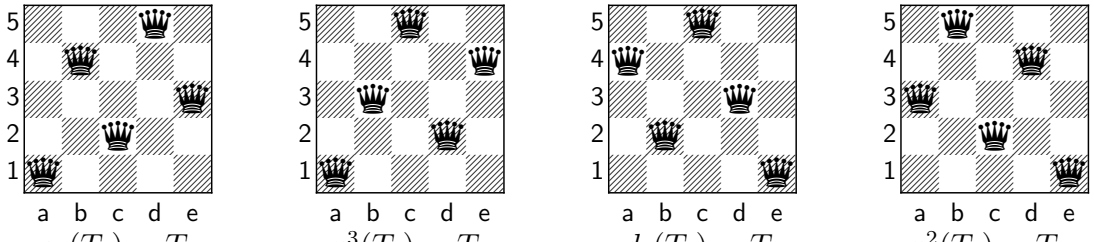

En efecte si $T_1$ i $T_2$ són respectivament les solucions anteriors aleshores:

$$Id(T_1) = r(T_1) = r^2(T_1) = r^3(T_1) = T_1,\ s_h(T_1) = s_v(T_1) = d_1(T_1) = d_2(T_1) = T_2$$

Amb òrbita de 8 elements:




$T_3$ $\quad d_1(T_3) = T_4$ $\quad r(T_3) = T_5$ $\quad s_v(T_3) = T_6$




$s_h(T_3) = T_7$ $\quad r^3(T_3) = T_8$ $\quad d_2(T_3) = T_9$ $\quad r^2(T_3) = T_{10}$

Les accions dels elements del grup diedral $D_{2\cdot 4}$ sobre les solucions al problema de les $n-$reines codificades per vectors $\vec{v} = (v_1, \ldots, v_n) \in S_{\{1,\ldots,n\}} \subseteq [1, \ldots, n]^n$, estan descrites a continuació (la demostració no és difícil):

$$\begin{array}{l} Id(v_1, \ldots, v_n) = (v_1, \ldots, v_n) \\ s_v(v_1, \ldots, v_n) = (v_n, \ldots, v_1) \\ s_h(v_1, \ldots, v_n) = (n+1-v_1, \ldots, n+1-v_n) \\ r^2(v_1, \ldots, v_n) = (n+1-v_n, \ldots, n+1-v_1) \end{array}$$

Si $\vec{u} := \sigma(\vec{v})$, on $\sigma \in \{r, r^3, d_1, d_2\}$ i les components del vector $\vec{v}$ prenen els valors $v_i = j$ aleshores les components del vector $\vec{u}$ prenen els següents valors

$$\begin{array}{lll} u_j = n+1-i & \text{si} & \sigma = r \\ u_{n+1-j} = i & \text{si} & \sigma = r^3 \\ u_{n+1-j} = n+1-i & \text{si} & \sigma = d_1 \\ u_j = i & \text{si} & \sigma = d_2 \end{array}$$

Es pot veure que si $r(\vec{u}) = \vec{u}$ aleshores $\vec{u}$ té una òrbita de 2 elements, si $r(\vec{u}) \neq \vec{u}$ i $r^2(\vec{u}) = \vec{u}$ aleshores $\vec{u}$ té una òrbita de 4 elements i en els altres casos $\vec{u}$ té una òrbita de 8 elements.

6. Calculeu, per tot $n \geq 2$, la descomposició de la matriu de solucions $MS$ en les matrius $M_2$, $M_4$ i $M_8$

$$MS = M_2 + M_4 + M_8$$

on $M_2$, $M_4$, $M_8$ conté per cada casella el nombre total de solucions amb òrbites de 2, 4 i 8 elements respectivament. Per valors baixos de $n$ tenim les següents descomposicions:

$$n = 4: \quad MS = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix} = M_2$$

$$n = 5: \quad MS = \begin{pmatrix} 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \\ 2 & 2 & 2 & 2 & 2 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 2 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{pmatrix} + \begin{pmatrix} 2 & 1 & 2 & 1 & 2 \\ 1 & 2 & 2 & 2 & 1 \\ 2 & 2 & 0 & 2 & 2 \\ 1 & 2 & 2 & 2 & 1 \\ 2 & 1 & 2 & 1 & 2 \end{pmatrix} = M_2 + M_8$$

$$n = 6: \quad MS = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 \end{pmatrix} = M_4$$

Observeu que per $n = 1$, l'única solució (1) té una òrbita de 1 element i per tant, en aquest cas, $MS = M_1 = (1)$.

7. Desa en un vector, que anomenarem vector de solucions essencials, el nombre total de les solucions essencials amb òrbites de 2, 4 i 8 elements respectivament

$$VS = (e_2, e_4, e_8), \quad (\forall n \geq 2)$$

i calculeu el nombre total de solucions essencials.

Per $n = 1$ el vector de solucions essencials és $VS = (e_1) = (1)$.

# Enunciat del treball

Cal fer un programa en C per trobar o analitzar solucions al problema de les $n-$reines. Es lliurarà un únic fitxer, anomenat `vuit_reines.c`

Aquest fitxer s'ha de poder compilar sense errors amb la instrucció

```
$ gcc -Wall -o vuit_reines vuit_reines.c -lm      (Linux / MacOs)
$ gcc -Wall -o vuit_reines.exe vuit_reines.c -lm  (Windows)
```

Les dades s'han de llegir de fitxers de text i els resultats també cal escriurel's en fitxers de text. El programa ha de contenir les 5 funcions que es proposen per trobar o analitzar solucions. L'usuari ha de poder seleccionar quina funció/algorisme farà servir i a quin tauler $n \times n$ cal aplicar-lo, mitjançant arguments de la funció main.

El fitxer executable obtingut s'ha de poder executar des de la consola, escrivint en la instrucció d'execució la mida del tauler d'escacs, un paràmetre que indiqui el nom de l'algorisme a usar, un fitxer de text d'entrada i un fitxer de text de sortida:

```
$ ./vuit_reines mida algorisme fitxer_entrada.txt fitxer_sortida.txt
  (Linux / MacOs)

$ .\vuit_reines.exe mida algorisme fitxer_entrada.txt fitxer_sortida.txt
  (Windows)
```

Els possibles paràmetres (algorismes) que es poden escriure per executar el programa seran: aleatori1, aleatori2, aleatori3, backtrack i matriu.

El fitxer d'entrada només és necessari en el primer algorisme (aleatori1) i en el darrer algorisme (matriu), tot i així en les intruccions d'execució escriurem un fitxer anomenat `entrada.txt` de contingut buit per la resta d'algorismes: aleatori2, aleatori3, backtrack.

En la resta dels fitxers d'entrada (aleatori1 i matriu) i en tots els fitxers de sortida indicarem l'algorisme aplicat i la mida del tauler.

En el cas que no tingui solució (casos $n = 2, n = 3$), el programa ha de donar un missatge d'error

```
$ ./vuit_reines 3 backtrack entrada.txt sortida_bactrack_3x3.txt
  (Linux / MacOs)

$ .\vuit_reines.exe 3 backtrack entrada.txt sortida_bactrack_3x3.txt
  (Windows)
Error: el tauler 3x3 no te solucio.
```

Exemples de com s'escriurà la instrucció d'execució:

```
$ ./vuit_reines 4 aleatori1 entrada_aleatori1_4x4_ns1.txt
  sortida_aleatori1_4x4_ns1.txt
  (Linux / MacOs)
$ .\vuit_reines.exe 4 aleatori1 entrada_aleatori1_4x4_ns1.txt
  sortida_aleatori1_4x4_ns1.txt
  (Windows)

$ ./vuit_reines 5 aleatori2 entrada.txt sortida_aleatori2_5x5_s1.txt
  (Linux / MacOs)
$ .\vuit_reines.exe 5 aleatori2 entrada.txt sortida_aleatori2_5x5_s1.txt
  (Windows)

$ ./vuit_reines 6 aleatori3 entrada.txt sortida_aleatori3_6x6.txt
  (Linux / MacOs)
$ .\vuit_reines.exe 6 aleatori3 entrada.txt sortida_aleatori3_6x6.txt
  (Windows)

$ ./vuit_reines 7 backtrack entrada.txt sortida_bactrack_7x7.txt
  (Linux / MacOs)
$ .\vuit_reines.exe 7 backtrack entrada.txt sortida_bactrack_7x7.txt
  (Windows)

$ ./vuit_reines 8 matriu entrada_matriu.txt sortida_matriu_8x8.txt
  (Linux / MacOs)
$ .\vuit_reines.exe 8 matriu entrada_matriu.txt sortida_matriu_8x8.txt
  (Windows)
```

On `s1` vol dir que el vector que conté el fitxer és solució i `ns1` vol dir no ho és.

Les úniques impressions per pantalla seran la mida del tauler que s'analitza, l'algorisme utilitzat entre parèntesis i:

- per la funció `es_solucio` (aleatori1): el vector que es proposa i la decisió de la funció (és solució o no és solució).

- per la funció `vector_aleatori` (aleatori2): el vector que calcula la funció i la decisió de la funció `es_solucio` (és solució o no és solució).

- per la funció `totes_les_solucions_aletories` (aleatori3): totes les solucions al problema de les $n-$reines.

- per la funció `backtracking` (backtrack): el nombre total de solucions calculat per aquesta funció.

- per la funció `matriu_compta_solucions` (matriu): la matriu MS, les matrius M2, M4 i M8, el vector de solucions essencials VS i el nombre total de solucions essencials.

En els algorismes aleatoris s'utilitzarà la funció `rand()` de C per generar nombres pseudoaleatoris i han de decidir si els vectors propossat són solució o no. Es valorarà que s'inicialitzi el generador de nombres pseudoaleatoris amb `srand()`.

L'algorisme backtracking es basarà en el pseudocodi que proposem.

A l'inici del fitxer, hi haurà d'haver un bloc de línies (comentaris de C) amb informació sobre l'autor i el programa en si. En aquesta informació podeu fer un resum dels resultats obtinguts amb el vostre codi. (màxim de 20 línies)

# Avaluació

El programa haurà d'estar estructurat en funcions, evitant les variables globals tant com sigui possible. En valorar la implementació de cada funció, el fet de no tenir una estructura clara o d'usar variables globals faria que la puntuació fos menor.

El programa no ha de llegir cap valor amb la funció `scanf`.

# Programa base i algorisme aleatori: 3 punts

- Interpretar correctament els paràmetres del main i tractar correctament la sortida en el format demanat: 1 punt
- Llegir correctament els fitxers d'entrada i tractament dels fitxers de sortida amb la informació dels taulers: 1 punt
- Implementació de les funcions dels 3 primers apartats: 1 punt

# Algorisme backtracking: 3 punts

- Programació correcta de `tauler` tant si és un array (vector) com una llista o una pila: 1 punt
- Implementació recursiva del backtracking segons el pseudocodi donat: 1,5 punt
- Tractament correcte de les solucions: 0,5 punts

# Matriu de solucions: 3 punts

- Càlcul de la matriu de solucions: 1 punt
- Descomposició de la matriu de solucions en les matrius M2, M4, M8: 1 punt
- Vector de solucions essencials i nombre total de solucions essencials: 1 punt

# Altres: 1 punt

- Velocitat del programa (mida dels taulers que és capaç de resoldre en un temps fixat): 0,5 punts
- Reserva de memòria amb assignació dinàmica: 0,5 punts

## Errors i warnings

El fitxer es compilarà amb la instrucció

```
$ gcc -Wall -o vuit_reines vuit_reines.c -lm      (Linux / MacOs)
$ gcc -Wall -o vuit_reines.exe vuit_reines.c -lm  (Windows)
```

Si la compilació produeix errors i no genera un fitxer executable la nota serà un zero.

Si la compilació genera un fitxer executable i s'executa correctament però dona advertències (warnings), cada advertència de tipus diferent restarà 0,2 punts a la nota del treball.

## Elaboració individual del treball

Podeu inspirar-vos en recursos que trobeu per internet o en converses amb companys, però haureu d'explicar en una entrevista com funciona el codi i no serà valorat si poseu codi que no enteneu. No compartiu codi: entregar una part significativa del programa igual que un company es puntua amb un zero del treball (i per tant un suspens a l'assignatura).

## Data límit

31 de maig a les 23:59